

# SIMATS ENGINEERING

## ASSESSMENT TOOL 2 — CASE STUDY

**Course Name:** Computer Networks

**Course Code:** CSA07

**Course Outcome:** CO3 — Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions

### Question 1: Video Conferencing Platform — Packet Loss, Jitter, and Delay

#### 1. Analysis: Transport Protocol vs Application Layer

The problem described — 8% packet loss, 180 ms jitter, frozen video frames, and voice distortion during peak hours — is primarily caused by the choice and behaviour of the transport protocol, though the application layer's buffering strategy amplifies the visible effect.

If the platform uses TCP for real-time audio/video, TCP's reliability mechanisms (retransmission, in-order delivery, congestion control) are the root cause. When a packet is lost, TCP halts delivery of all subsequent data until the lost segment is retransmitted and acknowledged. Under 8% loss and high jitter, this creates head-of-line blocking, so frames arrive too late to be useful, causing the frozen-frame symptom described.

If UDP is already being used (as most real conferencing platforms do), the packet loss and jitter are largely a symptom of network-layer congestion at peak hours (shared bandwidth, queuing delay, router buffer overflow) rather than the protocol itself. However, the application layer is still responsible for how it copes with this loss — if the jitter buffer or packet-loss concealment logic is poorly tuned, the same underlying network conditions will manifest as worse distortion and freezing.

Conclusion: the underlying trigger is transport/network-layer congestion and loss, but the severity experienced by users is strongly shaped by the application layer's buffering and concealment strategy. Both layers must be examined together.

#### 2. Justification Based on Real-Time Multimedia Requirements

Real-time multimedia communication has fundamentally different requirements from typical data transfer:

- Timeliness over completeness: a late packet is worse than a lost one, since stale audio/video frames are useless once the presentation deadline passes.
- Tolerance for loss: human perception can tolerate small amounts of packet loss (a brief glitch) far better than added delay.
- Low and predictable latency: interactive conversation requires end-to-end delay under roughly 150 ms to feel natural; jitter must be smoothed without adding excessive buffering delay.
- Continuous stream delivery: video/audio codecs expect a steady stream of samples; retransmission-based recovery disrupts this rhythm.

TCP's guarantees (in-order, lossless delivery) directly conflict with these requirements because retransmission and congestion-control back-off increase delay precisely when the network is already congested. UDP better matches these requirements because it delivers packets immediately without waiting for retransmission, leaving loss-handling and pacing to the application (via RTP/RTCP), which is the standard design used by real conferencing systems.

#### 3. Recommended Protocol-Level and Application-Level Improvements

##### Protocol-level:

- Use UDP as the transport, carried over RTP/RTCP (or SRTP for security) instead of TCP, so late data is simply dropped rather than blocking the stream.
- Enable adaptive bitrate/congestion control designed for real-time media, such as Google Congestion Control (GCC) or WebRTC's built-in bandwidth estimation, so encoding rate adjusts to available bandwidth instead of overflowing the network.
- Apply Differentiated Services (DiffServ/QoS) marking (e.g., EF — Expedited Forwarding) so routers prioritize voice/video packets over best-effort traffic during peak congestion.
- Use Forward Error Correction (FEC) to reconstruct a percentage of lost packets without needing retransmission, which is well suited to an 8% loss rate.

### Application-level:

- Implement an adaptive jitter buffer that dynamically resizes based on measured jitter (180 ms) rather than using a fixed buffer, balancing smoothness against added delay.
- Use packet-loss concealment (PLC) algorithms in the audio codec (e.g., Opus's built-in PLC) to mask short losses without perceptible artifacts.
- Enable simulcast/SVC (Scalable Video Coding) so the server can drop video quality layers for congested clients instead of freezing the whole stream.
- Monitor QoE metrics (loss, jitter, RTT) continuously and dynamically switch media servers or routes (using a geographically distributed SFU/media-relay architecture) to reduce peak-hour congestion.

## Question 2: High DNS Lookup Time in a Multi-Continent Web Service

### 1. Reasons Behind the High DNS Resolution Time ( $\approx 800$ ms)

- Centralized/single-location DNS servers: if authoritative or resolver infrastructure is concentrated in one region, users far from that location incur significant propagation delay for every query.
- Deep recursive resolution chain: each lookup may need to traverse root  $\rightarrow$  TLD  $\rightarrow$  authoritative name servers if results are not cached, adding multiple round trips.
- Low cache hit ratio: aggressive or very low TTL values force frequent re-resolution instead of serving from local resolver cache.
- Lack of geographically distributed name servers: without Anycast, all queries funnel to the same physical server regardless of the user's location, adding queuing delay under load.
- No local DNS caching/pre-fetching at the client or ISP resolver level, so even repeat lookups pay the full round-trip cost.
- Overloaded or under-provisioned DNS servers during peak traffic, increasing queuing and processing delay.

### 2. Proposed DNS Architecture to Reduce Lookup Time Below 50 ms

**Anycast DNS:** Deploy the same authoritative DNS IP address from multiple geographically distributed points of presence (PoPs). BGP routing automatically directs each user's query to the nearest/lowest-latency PoP, eliminating long-haul propagation delay and providing built-in load distribution and failover.

**DNS Pre-fetching:** Have client applications and browsers resolve domain names for likely-needed resources in advance (e.g., while a user is on a login page, pre-resolve the domains of resources the homepage will need), so the resolution is already complete by the time the request is made.

**TTL Optimization:** Set TTL values that balance cache freshness against lookup frequency — moderate TTLs (e.g., 300–3600 seconds) for records that rarely change, allowing recursive resolvers and clients to serve repeat queries from cache without additional round trips, while keeping TTLs low enough to support failover when needed.

Combined effect: Anycast reduces the physical distance/latency per query, pre-fetching removes resolution from the critical path of page load, and TTL tuning maximizes cache reuse — together bringing typical lookups from ~800 ms down to well under 50 ms for cached/nearby queries.

### 3. Advantages and Limitations of the Proposed Architecture

#### Advantages:

- Performance: Anycast routes queries to the nearest server, sharply cutting round-trip time; pre-fetching hides latency from the user entirely.
- Scalability: multiple Anycast PoPs can each absorb a share of global query load, and the architecture scales horizontally by adding more PoPs.
- Availability: if one PoP fails, BGP automatically reroutes traffic to the next-nearest healthy PoP, providing seamless failover without manual DNS changes.

#### Limitations:

- Anycast routing depends on BGP convergence; misconfigured or unstable routing can occasionally send queries to a suboptimal or distant PoP.
- Overly long TTLs improve performance but slow down propagation of legitimate DNS changes (e.g., during failover or migration), while overly short TTLs increase load and reduce the performance benefit.
- Pre-fetching consumes extra client and network resources for resolutions that may not always be used, and requires careful implementation to avoid privacy/tracking concerns.
- Operating multiple Anycast PoPs increases infrastructure cost and operational complexity (monitoring, synchronization of DNS records across sites).

## Question 3: Centralized DNS Bottleneck for a Global Cloud Provider

### 4. Impact of Centralized DNS Architecture on Application Performance

Routing every global DNS query to a single primary server creates a single point of both latency and failure:

- High latency for distant users: users far from the primary server pay a large propagation delay on every uncached lookup (observed here as ~800 ms).
- Poor scalability under load: during promotional events, the sudden surge in simultaneous users creates queuing delay and possible timeouts as the single server becomes saturated.
- Single point of failure: if the primary server becomes unreachable or overloaded, all name resolution — and therefore all application access — fails globally, not just in one region.
- Cascading application-level delay: since DNS resolution is typically the first step before any HTTP/TCP connection, this bottleneck delays every downstream request, amplifying perceived slowness across the entire platform.

### 5. Design of a Distributed DNS Solution

A distributed, highly available DNS solution should combine the following components:

- Multiple Anycast-enabled authoritative DNS clusters deployed across continents (e.g., North America, Europe, Asia-Pacific), all announcing the same IP via BGP so each user reaches the nearest cluster.
- A tiered resolver hierarchy with geographically local recursive resolvers/edge caches close to end users, reducing dependence on any single origin server.

- Health-check-driven failover so that if one regional cluster becomes unhealthy, BGP/health checks redirect traffic to the next-nearest healthy cluster automatically.
- Load-balanced secondary/replica name servers within each region to absorb traffic spikes such as promotional events, using zone transfers (AXFR/IXFR) to keep records synchronized.
- Monitoring and analytics on query volume and latency per region to proactively scale capacity ahead of anticipated high-traffic events.

This design removes the single point of failure, distributes load across many servers instead of one, and places DNS resolution physically closer to users worldwide.

## 6. Influence of TTL, Anycast Routing, and DNS Caching on Performance and Fault Tolerance

**TTL values:** Moderate TTLs allow resolvers to cache answers and avoid repeated queries, reducing average latency and server load; however, TTLs that are too long delay the propagation of failover changes, while TTLs that are too short increase query volume and reduce the benefit of caching — the value must be tuned per record type (e.g., shorter TTLs for records tied to failover, longer for stable records).

**Anycast routing:** By allowing the same IP address to be served from many locations, Anycast reduces per-query latency (queries go to the nearest instance) and improves fault tolerance (BGP reroutes automatically around a failed or unreachable site) and naturally load-balances traffic across sites during demand spikes.

**DNS caching:** Caching at recursive resolvers, ISPs, and client devices means most repeat queries are answered locally without contacting the authoritative servers at all, dramatically cutting both latency and load on the origin infrastructure — but stale cached entries (due to overly long TTLs) can cause clients to keep contacting a failed server until the cache expires.

Together, these three mechanisms trade off responsiveness, freshness, and resilience; a well-tuned combination gives low latency in the common case while still allowing timely failover when something goes wrong.

## Question 4: TCP Latency Spike in an Electronic Stock Trading Platform

### 1. Three TCP-Related Factors Responsible for the Latency Increase

**a) Flow Control:** TCP's receive-window mechanism limits how much unacknowledged data a sender can transmit. Under a sudden surge of orders at market open, if the receiver (server) cannot process and acknowledge data fast enough, its advertised window shrinks, forcing the sender to pause transmission — introducing delay even though the network itself has capacity.

**b) Nagle's Algorithm:** Nagle's algorithm delays sending small packets so they can be coalesced into larger ones, reducing overhead for bulk transfers. For a trading system that sends many small, latency-sensitive order messages, this delay (which can combine badly with delayed ACKs, producing pauses of up to ~200 ms) is directly counterproductive, since each order needs to be sent immediately rather than batched.

**c) SYN Queue Limitations:** Each new TCP connection begins with a three-way handshake, and incoming SYN requests are held in a fixed-size SYN queue (backlog) at the server before being accepted. During the burst of connection attempts at market opening, if the SYN queue is too small or the accept rate too slow, new connection requests queue up or are dropped and retried, adding to the connection-establishment latency seen in the reported figures.

### 2. How Each Factor Affects High-Frequency Transaction Processing

- Flow control: causes intermittent stalls in already-open connections precisely when order volume is highest, meaning some orders sit buffered rather than being transmitted, directly increasing acknowledgment latency from the reported 1 ms baseline.

- Nagle's algorithm: delays the transmission of individual small order/acknowledgment messages while waiting to batch them, which is fundamentally at odds with a system where each message must be sent with minimal delay — this alone can account for tens of milliseconds of added latency per message.
- SYN queue limitations: increase the time needed to establish new sessions during the burst of market-open activity, so trading clients experience connection delays before they can even begin submitting orders, and repeated SYN retransmissions after drops further inflate the observed latency.

### 3. Recommended TCP Configuration Changes

- Disable Nagle's algorithm (set `TCP_NODELAY`) on all trading connections so small order messages are transmitted immediately rather than buffered for coalescing.
- Disable or tune delayed ACKs (avoid the classic Nagle + delayed-ACK interaction) so acknowledgments are also sent promptly rather than being held back.
- Increase the TCP receive/send window sizes (and enable window scaling) so flow control does not unnecessarily throttle throughput during high-order-volume bursts.
- Increase the SYN backlog queue size (e.g., tuning `somaxconn`/`tcp_max_syn_backlog``) and enable SYN cookies, so the server can absorb a burst of new connection requests at market open without dropping or delaying handshakes.
- Use persistent/pre-established (keep-alive) connections for known trading clients instead of opening a new TCP connection per session, avoiding repeated handshake overhead entirely.
- Consider a low-latency transport alternative for the most latency-sensitive paths (e.g., UDP-based proprietary protocols with application-level reliability, or kernel-bypass techniques) where regulatory and reliability requirements permit.
- Continuously monitor retransmission rates, RTT, and queue depths, and auto-scale server capacity ahead of known high-volume periods such as market open.

## Question 1: Video Conferencing Platform — Packet Loss, Jitter, and Delay

### 1. Analysis: Transport Protocol vs Application Layer

The problem described — 8% packet loss, 180 ms jitter, frozen video frames, and voice distortion during peak hours — is primarily caused by the choice and behaviour of the transport protocol, though the application layer's buffering strategy amplifies the visible effect.

If the platform uses TCP for real-time audio/video, TCP's reliability mechanisms (retransmission, in-order delivery, congestion control) are the root cause. When a packet is lost, TCP halts delivery of all subsequent data until the lost segment is retransmitted and acknowledged. Under 8% loss and high jitter, this creates head-of-line blocking, so frames arrive too late to be useful, causing the frozen-frame symptom described.

If UDP is already being used (as most real conferencing platforms do), the packet loss and jitter are largely a symptom of network-layer congestion at peak hours (shared bandwidth, queuing delay, router buffer overflow) rather than the protocol itself. However, the application layer is still responsible for how it copes with this loss — if the jitter buffer or packet-loss concealment logic is poorly tuned, the same underlying network conditions will manifest as worse distortion and freezing.

Conclusion: the underlying trigger is transport/network-layer congestion and loss, but the severity experienced by users is strongly shaped by the application layer's buffering and concealment strategy. Both layers must be examined together.

### 2. Justification Based on Real-Time Multimedia Requirements

Real-time multimedia communication has fundamentally different requirements from typical data transfer:

- Timeliness over completeness: a late packet is worse than a lost one, since stale audio/video frames are useless once the presentation deadline passes.
- Tolerance for loss: human perception can tolerate small amounts of packet loss (a brief glitch) far better than added delay.
- Low and predictable latency: interactive conversation requires end-to-end delay under roughly 150 ms to feel natural; jitter must be smoothed without adding excessive buffering delay.
- Continuous stream delivery: video/audio codecs expect a steady stream of samples; retransmission-based recovery disrupts this rhythm.

TCP's guarantees (in-order, lossless delivery) directly conflict with these requirements because retransmission and congestion-control back-off increase delay precisely when the network is already congested. UDP better matches these requirements because it delivers packets immediately without waiting for retransmission, leaving loss-handling and pacing to the application (via RTP/RTCP), which is the standard design used by real conferencing systems.

### 3. Recommended Protocol-Level and Application-Level Improvements

#### Protocol-level:

- Use UDP as the transport, carried over RTP/RTCP (or SRTP for security) instead of TCP, so late data is simply dropped rather than blocking the stream.
- Enable adaptive bitrate/congestion control designed for real-time media, such as Google Congestion Control (GCC) or WebRTC's built-in bandwidth estimation, so encoding rate adjusts to available bandwidth instead of overflowing the network.
- Apply Differentiated Services (DiffServ/QoS) marking (e.g., EF — Expedited Forwarding) so routers prioritize voice/video packets over best-effort traffic during peak congestion.
- Use Forward Error Correction (FEC) to reconstruct a percentage of lost packets without needing retransmission, which is well suited to an 8% loss rate.

#### Application-level:

- Implement an adaptive jitter buffer that dynamically resizes based on measured jitter (180 ms) rather than using a fixed buffer, balancing smoothness against added delay.
- Use packet-loss concealment (PLC) algorithms in the audio codec (e.g., Opus's built-in PLC) to mask short losses without perceptible artifacts.
- Enable simulcast/SVC (Scalable Video Coding) so the server can drop video quality layers for congested clients instead of freezing the whole stream.
- Monitor QoE metrics (loss, jitter, RTT) continuously and dynamically switch media servers or routes (using a geographically distributed SFU/media-relay architecture) to reduce peak-hour congestion.

## Question 2: High DNS Lookup Time in a Multi-Continent Web Service

### 1. Reasons Behind the High DNS Resolution Time ( $\approx 800$ ms)

- Centralized/single-location DNS servers: if authoritative or resolver infrastructure is concentrated in one region, users far from that location incur significant propagation delay for every query.
- Deep recursive resolution chain: each lookup may need to traverse root  $\rightarrow$  TLD  $\rightarrow$  authoritative name servers if results are not cached, adding multiple round trips.
- Low cache hit ratio: aggressive or very low TTL values force frequent re-resolution instead of serving from local resolver cache.



- Lack of geographically distributed name servers: without Anycast, all queries funnel to the same physical server regardless of the user's location, adding queuing delay under load.
- No local DNS caching/pre-fetching at the client or ISP resolver level, so even repeat lookups pay the full round-trip cost.
- Overloaded or under-provisioned DNS servers during peak traffic, increasing queuing and processing delay.

## 2. Proposed DNS Architecture to Reduce Lookup Time Below 50 ms

**Anycast DNS:** Deploy the same authoritative DNS IP address from multiple geographically distributed points of presence (PoPs). BGP routing automatically directs each user's query to the nearest/lowest-latency PoP, eliminating long-haul propagation delay and providing built-in load distribution and failover.

**DNS Pre-fetching:** Have client applications and browsers resolve domain names for likely-needed resources in advance (e.g., while a user is on a login page, pre-resolve the domains of resources the homepage will need), so the resolution is already complete by the time the request is made.

**TTL Optimization:** Set TTL values that balance cache freshness against lookup frequency — moderate TTLs (e.g., 300–3600 seconds) for records that rarely change, allowing recursive resolvers and clients to serve repeat queries from cache without additional round trips, while keeping TTLs low enough to support failover when needed.

Combined effect: Anycast reduces the physical distance/latency per query, pre-fetching removes resolution from the critical path of page load, and TTL tuning maximizes cache reuse — together bringing typical lookups from ~800 ms down to well under 50 ms for cached/nearby queries.

## 3. Advantages and Limitations of the Proposed Architecture

### Advantages:

- Performance: Anycast routes queries to the nearest server, sharply cutting round-trip time; pre-fetching hides latency from the user entirely.
- Scalability: multiple Anycast PoPs can each absorb a share of global query load, and the architecture scales horizontally by adding more PoPs.
- Availability: if one PoP fails, BGP automatically reroutes traffic to the next-nearest healthy PoP, providing seamless failover without manual DNS changes.

### Limitations:

- Anycast routing depends on BGP convergence; misconfigured or unstable routing can occasionally send queries to a suboptimal or distant PoP.
- Overly long TTLs improve performance but slow down propagation of legitimate DNS changes (e.g., during failover or migration), while overly short TTLs increase load and reduce the performance benefit.
- Pre-fetching consumes extra client and network resources for resolutions that may not always be used, and requires careful implementation to avoid privacy/tracking concerns.
- Operating multiple Anycast PoPs increases infrastructure cost and operational complexity (monitoring, synchronization of DNS records across sites).

## Question 3: Centralized DNS Bottleneck for a Global Cloud Provider

### 4. Impact of Centralized DNS Architecture on Application Performance

Routing every global DNS query to a single primary server creates a single point of both latency and failure:

- High latency for distant users: users far from the primary server pay a large propagation delay on every uncached lookup (observed here as ~800 ms).
- Poor scalability under load: during promotional events, the sudden surge in simultaneous users creates queuing delay and possible timeouts as the single server becomes saturated.
- Single point of failure: if the primary server becomes unreachable or overloaded, all name resolution — and therefore all application access — fails globally, not just in one region.
- Cascading application-level delay: since DNS resolution is typically the first step before any HTTP/TCP connection, this bottleneck delays every downstream request, amplifying perceived slowness across the entire platform.

## 5. Design of a Distributed DNS Solution

A distributed, highly available DNS solution should combine the following components:

- Multiple Anycast-enabled authoritative DNS clusters deployed across continents (e.g., North America, Europe, Asia-Pacific), all announcing the same IP via BGP so each user reaches the nearest cluster.
- A tiered resolver hierarchy with geographically local recursive resolvers/edge caches close to end users, reducing dependence on any single origin server.
- Health-check-driven failover so that if one regional cluster becomes unhealthy, BGP/health checks redirect traffic to the next-nearest healthy cluster automatically.
- Load-balanced secondary/replica name servers within each region to absorb traffic spikes such as promotional events, using zone transfers (AXFR/IXFR) to keep records synchronized.
- Monitoring and analytics on query volume and latency per region to proactively scale capacity ahead of anticipated high-traffic events.

This design removes the single point of failure, distributes load across many servers instead of one, and places DNS resolution physically closer to users worldwide.

## 6. Influence of TTL, Anycast Routing, and DNS Caching on Performance and Fault Tolerance

**TTL values:** Moderate TTLs allow resolvers to cache answers and avoid repeated queries, reducing average latency and server load; however, TTLs that are too long delay the propagation of failover changes, while TTLs that are too short increase query volume and reduce the benefit of caching — the value must be tuned per record type (e.g., shorter TTLs for records tied to failover, longer for stable records).

**Anycast routing:** By allowing the same IP address to be served from many locations, Anycast reduces per-query latency (queries go to the nearest instance) and improves fault tolerance (BGP reroutes automatically around a failed or unreachable site) and naturally load-balances traffic across sites during demand spikes.

**DNS caching:** Caching at recursive resolvers, ISPs, and client devices means most repeat queries are answered locally without contacting the authoritative servers at all, dramatically cutting both latency and load on the origin infrastructure — but stale cached entries (due to overly long TTLs) can cause clients to keep contacting a failed server until the cache expires.

Together, these three mechanisms trade off responsiveness, freshness, and resilience; a well-tuned combination gives low latency in the common case while still allowing timely failover when something goes wrong.

## Question 4: TCP Latency Spike in an Electronic Stock Trading Platform

### 1. Three TCP-Related Factors Responsible for the Latency Increase

**a) Flow Control:** TCP's receive-window mechanism limits how much unacknowledged data a sender can transmit. Under a sudden surge of orders at market open, if the receiver (server) cannot process and



acknowledge data fast enough, its advertised window shrinks, forcing the sender to pause transmission — introducing delay even though the network itself has capacity.

**b) Nagle's Algorithm:** Nagle's algorithm delays sending small packets so they can be coalesced into larger ones, reducing overhead for bulk transfers. For a trading system that sends many small, latency-sensitive order messages, this delay (which can combine badly with delayed ACKs, producing pauses of up to ~200 ms) is directly counterproductive, since each order needs to be sent immediately rather than batched.

**c) SYN Queue Limitations:** Each new TCP connection begins with a three-way handshake, and incoming SYN requests are held in a fixed-size SYN queue (backlog) at the server before being accepted. During the burst of connection attempts at market opening, if the SYN queue is too small or the accept rate too slow, new connection requests queue up or are dropped and retried, adding to the connection-establishment latency seen in the reported figures.

## 2. How Each Factor Affects High-Frequency Transaction Processing

- Flow control: causes intermittent stalls in already-open connections precisely when order volume is highest, meaning some orders sit buffered rather than being transmitted, directly increasing acknowledgment latency from the reported 1 ms baseline.
- Nagle's algorithm: delays the transmission of individual small order/acknowledgment messages while waiting to batch them, which is fundamentally at odds with a system where each message must be sent with minimal delay — this alone can account for tens of milliseconds of added latency per message.
- SYN queue limitations: increase the time needed to establish new sessions during the burst of market-open activity, so trading clients experience connection delays before they can even begin submitting orders, and repeated SYN retransmissions after drops further inflate the observed latency.

## 3. Recommended TCP Configuration Changes

- Disable Nagle's algorithm (set `TCP_NODELAY`) on all trading connections so small order messages are transmitted immediately rather than buffered for coalescing.
- Disable or tune delayed ACKs (avoid the classic Nagle + delayed-ACK interaction) so acknowledgments are also sent promptly rather than being held back.
- Increase the TCP receive/send window sizes (and enable window scaling) so flow control does not unnecessarily throttle throughput during high-order-volume bursts.
- Increase the SYN backlog queue size (e.g., tuning `'somaxconn'/'tcp_max_syn_backlog'`) and enable SYN cookies, so the server can absorb a burst of new connection requests at market open without dropping or delaying handshakes.
- Use persistent/pre-established (keep-alive) connections for known trading clients instead of opening a new TCP connection per session, avoiding repeated handshake overhead entirely.
- Consider a low-latency transport alternative for the most latency-sensitive paths (e.g., UDP-based proprietary protocols with application-level reliability, or kernel-bypass techniques) where regulatory and reliability requirements permit.
- Continuously monitor retransmission rates, RTT, and queue depths, and auto-scale server capacity ahead of known high-volume periods such as market open.

## Question 1: Video Conferencing Platform — Packet Loss, Jitter, and Delay

### 1. Analysis: Transport Protocol vs Application Layer

The problem described — 8% packet loss, 180 ms jitter, frozen video frames, and voice distortion during peak hours — is primarily caused by the choice and behaviour of the transport protocol, though the application layer's buffering strategy amplifies the visible effect.

If the platform uses TCP for real-time audio/video, TCP's reliability mechanisms (retransmission, in-order delivery, congestion control) are the root cause. When a packet is lost, TCP halts delivery of all subsequent data until the lost segment is retransmitted and acknowledged. Under 8% loss and high jitter, this creates head-of-line blocking, so frames arrive too late to be useful, causing the frozen-frame symptom described.

If UDP is already being used (as most real conferencing platforms do), the packet loss and jitter are largely a symptom of network-layer congestion at peak hours (shared bandwidth, queuing delay, router buffer overflow) rather than the protocol itself. However, the application layer is still responsible for how it copes with this loss — if the jitter buffer or packet-loss concealment logic is poorly tuned, the same underlying network conditions will manifest as worse distortion and freezing.

Conclusion: the underlying trigger is transport/network-layer congestion and loss, but the severity experienced by users is strongly shaped by the application layer's buffering and concealment strategy. Both layers must be examined together.

## 2. Justification Based on Real-Time Multimedia Requirements

Real-time multimedia communication has fundamentally different requirements from typical data transfer:

- Timeliness over completeness: a late packet is worse than a lost one, since stale audio/video frames are useless once the presentation deadline passes.
- Tolerance for loss: human perception can tolerate small amounts of packet loss (a brief glitch) far better than added delay.
- Low and predictable latency: interactive conversation requires end-to-end delay under roughly 150 ms to feel natural; jitter must be smoothed without adding excessive buffering delay.
- Continuous stream delivery: video/audio codecs expect a steady stream of samples; retransmission-based recovery disrupts this rhythm.

TCP's guarantees (in-order, lossless delivery) directly conflict with these requirements because retransmission and congestion-control back-off increase delay precisely when the network is already congested. UDP better matches these requirements because it delivers packets immediately without waiting for retransmission, leaving loss-handling and pacing to the application (via RTP/RTCP), which is the standard design used by real conferencing systems.

## 3. Recommended Protocol-Level and Application-Level Improvements

### Protocol-level:

- Use UDP as the transport, carried over RTP/RTCP (or SRTP for security) instead of TCP, so late data is simply dropped rather than blocking the stream.
- Enable adaptive bitrate/congestion control designed for real-time media, such as Google Congestion Control (GCC) or WebRTC's built-in bandwidth estimation, so encoding rate adjusts to available bandwidth instead of overflowing the network.
- Apply Differentiated Services (DiffServ/QoS) marking (e.g., EF — Expedited Forwarding) so routers prioritize voice/video packets over best-effort traffic during peak congestion.
- Use Forward Error Correction (FEC) to reconstruct a percentage of lost packets without needing retransmission, which is well suited to an 8% loss rate.

### Application-level:

- Implement an adaptive jitter buffer that dynamically resizes based on measured jitter (180 ms) rather than using a fixed buffer, balancing smoothness against added delay.
- Use packet-loss concealment (PLC) algorithms in the audio codec (e.g., Opus's built-in PLC) to mask short losses without perceptible artifacts.
- Enable simulcast/SVC (Scalable Video Coding) so the server can drop video quality layers for congested clients instead of freezing the whole stream.

- Monitor QoE metrics (loss, jitter, RTT) continuously and dynamically switch media servers or routes (using a geographically distributed SFU/media-relay architecture) to reduce peak-hour congestion.

## Question 2: High DNS Lookup Time in a Multi-Continent Web Service

### 1. Reasons Behind the High DNS Resolution Time ( $\approx 800$ ms)

- Centralized/single-location DNS servers: if authoritative or resolver infrastructure is concentrated in one region, users far from that location incur significant propagation delay for every query.
- Deep recursive resolution chain: each lookup may need to traverse root  $\rightarrow$  TLD  $\rightarrow$  authoritative name servers if results are not cached, adding multiple round trips.
- Low cache hit ratio: aggressive or very low TTL values force frequent re-resolution instead of serving from local resolver cache.
- Lack of geographically distributed name servers: without Anycast, all queries funnel to the same physical server regardless of the user's location, adding queuing delay under load.
- No local DNS caching/pre-fetching at the client or ISP resolver level, so even repeat lookups pay the full round-trip cost.
- Overloaded or under-provisioned DNS servers during peak traffic, increasing queuing and processing delay.

### 2. Proposed DNS Architecture to Reduce Lookup Time Below 50 ms

**Anycast DNS:** Deploy the same authoritative DNS IP address from multiple geographically distributed points of presence (PoPs). BGP routing automatically directs each user's query to the nearest/lowest-latency PoP, eliminating long-haul propagation delay and providing built-in load distribution and failover.

**DNS Pre-fetching:** Have client applications and browsers resolve domain names for likely-needed resources in advance (e.g., while a user is on a login page, pre-resolve the domains of resources the homepage will need), so the resolution is already complete by the time the request is made.

**TTL Optimization:** Set TTL values that balance cache freshness against lookup frequency — moderate TTLs (e.g., 300–3600 seconds) for records that rarely change, allowing recursive resolvers and clients to serve repeat queries from cache without additional round trips, while keeping TTLs low enough to support failover when needed.

Combined effect: Anycast reduces the physical distance/latency per query, pre-fetching removes resolution from the critical path of page load, and TTL tuning maximizes cache reuse — together bringing typical lookups from  $\sim 800$  ms down to well under 50 ms for cached/nearby queries.

### 3. Advantages and Limitations of the Proposed Architecture

#### Advantages:

- Performance: Anycast routes queries to the nearest server, sharply cutting round-trip time; pre-fetching hides latency from the user entirely.
- Scalability: multiple Anycast PoPs can each absorb a share of global query load, and the architecture scales horizontally by adding more PoPs.
- Availability: if one PoP fails, BGP automatically reroutes traffic to the next-nearest healthy PoP, providing seamless failover without manual DNS changes.

#### Limitations:

- Anycast routing depends on BGP convergence; misconfigured or unstable routing can occasionally send queries to a suboptimal or distant PoP.

- Overly long TTLs improve performance but slow down propagation of legitimate DNS changes (e.g., during failover or migration), while overly short TTLs increase load and reduce the performance benefit.
- Pre-fetching consumes extra client and network resources for resolutions that may not always be used, and requires careful implementation to avoid privacy/tracking concerns.
- Operating multiple Anycast PoPs increases infrastructure cost and operational complexity (monitoring, synchronization of DNS records across sites).

## Question 3: Centralized DNS Bottleneck for a Global Cloud Provider

### 4. Impact of Centralized DNS Architecture on Application Performance

Routing every global DNS query to a single primary server creates a single point of both latency and failure:

- High latency for distant users: users far from the primary server pay a large propagation delay on every uncached lookup (observed here as ~800 ms).
- Poor scalability under load: during promotional events, the sudden surge in simultaneous users creates queuing delay and possible timeouts as the single server becomes saturated.
- Single point of failure: if the primary server becomes unreachable or overloaded, all name resolution — and therefore all application access — fails globally, not just in one region.
- Cascading application-level delay: since DNS resolution is typically the first step before any HTTP/TCP connection, this bottleneck delays every downstream request, amplifying perceived slowness across the entire platform.

### 5. Design of a Distributed DNS Solution

A distributed, highly available DNS solution should combine the following components:

- Multiple Anycast-enabled authoritative DNS clusters deployed across continents (e.g., North America, Europe, Asia-Pacific), all announcing the same IP via BGP so each user reaches the nearest cluster.
- A tiered resolver hierarchy with geographically local recursive resolvers/edge caches close to end users, reducing dependence on any single origin server.
- Health-check-driven failover so that if one regional cluster becomes unhealthy, BGP/health checks redirect traffic to the next-nearest healthy cluster automatically.
- Load-balanced secondary/replica name servers within each region to absorb traffic spikes such as promotional events, using zone transfers (AXFR/IXFR) to keep records synchronized.
- Monitoring and analytics on query volume and latency per region to proactively scale capacity ahead of anticipated high-traffic events.

This design removes the single point of failure, distributes load across many servers instead of one, and places DNS resolution physically closer to users worldwide.

### 6. Influence of TTL, Anycast Routing, and DNS Caching on Performance and Fault Tolerance

**TTL values:** Moderate TTLs allow resolvers to cache answers and avoid repeated queries, reducing average latency and server load; however, TTLs that are too long delay the propagation of failover changes, while TTLs that are too short increase query volume and reduce the benefit of caching — the value must be tuned per record type (e.g., shorter TTLs for records tied to failover, longer for stable records).

**Anycast routing:** By allowing the same IP address to be served from many locations, Anycast reduces per-query latency (queries go to the nearest instance) and improves fault tolerance (BGP reroutes automatically around a failed or unreachable site) and naturally load-balances traffic across sites during demand spikes.

**DNS caching:** Caching at recursive resolvers, ISPs, and client devices means most repeat queries are answered locally without contacting the authoritative servers at all, dramatically cutting both latency and load on the origin infrastructure — but stale cached entries (due to overly long TTLs) can cause clients to keep contacting a failed server until the cache expires.

Together, these three mechanisms trade off responsiveness, freshness, and resilience; a well-tuned combination gives low latency in the common case while still allowing timely failover when something goes wrong.

## Question 4: TCP Latency Spike in an Electronic Stock Trading Platform

### 1. Three TCP-Related Factors Responsible for the Latency Increase

**a) Flow Control:** TCP's receive-window mechanism limits how much unacknowledged data a sender can transmit. Under a sudden surge of orders at market open, if the receiver (server) cannot process and acknowledge data fast enough, its advertised window shrinks, forcing the sender to pause transmission — introducing delay even though the network itself has capacity.

**b) Nagle's Algorithm:** Nagle's algorithm delays sending small packets so they can be coalesced into larger ones, reducing overhead for bulk transfers. For a trading system that sends many small, latency-sensitive order messages, this delay (which can combine badly with delayed ACKs, producing pauses of up to ~200 ms) is directly counterproductive, since each order needs to be sent immediately rather than batched.

**c) SYN Queue Limitations:** Each new TCP connection begins with a three-way handshake, and incoming SYN requests are held in a fixed-size SYN queue (backlog) at the server before being accepted. During the burst of connection attempts at market opening, if the SYN queue is too small or the accept rate too slow, new connection requests queue up or are dropped and retried, adding to the connection-establishment latency seen in the reported figures.

### 2. How Each Factor Affects High-Frequency Transaction Processing

- Flow control: causes intermittent stalls in already-open connections precisely when order volume is highest, meaning some orders sit buffered rather than being transmitted, directly increasing acknowledgment latency from the reported 1 ms baseline.
- Nagle's algorithm: delays the transmission of individual small order/acknowledgment messages while waiting to batch them, which is fundamentally at odds with a system where each message must be sent with minimal delay — this alone can account for tens of milliseconds of added latency per message.
- SYN queue limitations: increase the time needed to establish new sessions during the burst of market-open activity, so trading clients experience connection delays before they can even begin submitting orders, and repeated SYN retransmissions after drops further inflate the observed latency.

### 3. Recommended TCP Configuration Changes

- Disable Nagle's algorithm (set `TCP_NODELAY`) on all trading connections so small order messages are transmitted immediately rather than buffered for coalescing.
- Disable or tune delayed ACKs (avoid the classic Nagle + delayed-ACK interaction) so acknowledgments are also sent promptly rather than being held back.
- Increase the TCP receive/send window sizes (and enable window scaling) so flow control does not unnecessarily throttle throughput during high-order-volume bursts.
- Increase the SYN backlog queue size (e.g., tuning `somaxconn`/`tcp_max_syn_backlog``) and enable SYN cookies, so the server can absorb a burst of new connection requests at market open without dropping or delaying handshakes.
- Use persistent/pre-established (keep-alive) connections for known trading clients instead of opening a new TCP connection per session, avoiding repeated handshake overhead entirely.

- Consider a low-latency transport alternative for the most latency-sensitive paths (e.g., UDP-based proprietary protocols with application-level reliability, or kernel-bypass techniques) where regulatory and reliability requirements permit.
- Continuously monitor retransmission rates, RTT, and queue depths, and auto-scale server capacity ahead of known high-volume periods such as market open.

## Question 1: Video Conferencing Platform — Packet Loss, Jitter, and Delay

### 1. Analysis: Transport Protocol vs Application Layer

The problem described — 8% packet loss, 180 ms jitter, frozen video frames, and voice distortion during peak hours — is primarily caused by the choice and behaviour of the transport protocol, though the application layer's buffering strategy amplifies the visible effect.

If the platform uses TCP for real-time audio/video, TCP's reliability mechanisms (retransmission, in-order delivery, congestion control) are the root cause. When a packet is lost, TCP halts delivery of all subsequent data until the lost segment is retransmitted and acknowledged. Under 8% loss and high jitter, this creates head-of-line blocking, so frames arrive too late to be useful, causing the frozen-frame symptom described.

If UDP is already being used (as most real conferencing platforms do), the packet loss and jitter are largely a symptom of network-layer congestion at peak hours (shared bandwidth, queuing delay, router buffer overflow) rather than the protocol itself. However, the application layer is still responsible for how it copes with this loss — if the jitter buffer or packet-loss concealment logic is poorly tuned, the same underlying network conditions will manifest as worse distortion and freezing.

Conclusion: the underlying trigger is transport/network-layer congestion and loss, but the severity experienced by users is strongly shaped by the application layer's buffering and concealment strategy. Both layers must be examined together.

### 2. Justification Based on Real-Time Multimedia Requirements

Real-time multimedia communication has fundamentally different requirements from typical data transfer:

- Timeliness over completeness: a late packet is worse than a lost one, since stale audio/video frames are useless once the presentation deadline passes.
- Tolerance for loss: human perception can tolerate small amounts of packet loss (a brief glitch) far better than added delay.
- Low and predictable latency: interactive conversation requires end-to-end delay under roughly 150 ms to feel natural; jitter must be smoothed without adding excessive buffering delay.
- Continuous stream delivery: video/audio codecs expect a steady stream of samples; retransmission-based recovery disrupts this rhythm.

TCP's guarantees (in-order, lossless delivery) directly conflict with these requirements because retransmission and congestion-control back-off increase delay precisely when the network is already congested. UDP better matches these requirements because it delivers packets immediately without waiting for retransmission, leaving loss-handling and pacing to the application (via RTP/RTCP), which is the standard design used by real conferencing systems.

### 3. Recommended Protocol-Level and Application-Level Improvements

#### Protocol-level:

- Use UDP as the transport, carried over RTP/RTCP (or SRTP for security) instead of TCP, so late data is simply dropped rather than blocking the stream.



- Enable adaptive bitrate/congestion control designed for real-time media, such as Google Congestion Control (GCC) or WebRTC's built-in bandwidth estimation, so encoding rate adjusts to available bandwidth instead of overflowing the network.
- Apply Differentiated Services (DiffServ/QoS) marking (e.g., EF — Expedited Forwarding) so routers prioritize voice/video packets over best-effort traffic during peak congestion.
- Use Forward Error Correction (FEC) to reconstruct a percentage of lost packets without needing retransmission, which is well suited to an 8% loss rate.

### Application-level:

- Implement an adaptive jitter buffer that dynamically resizes based on measured jitter (180 ms) rather than using a fixed buffer, balancing smoothness against added delay.
- Use packet-loss concealment (PLC) algorithms in the audio codec (e.g., Opus's built-in PLC) to mask short losses without perceptible artifacts.
- Enable simulcast/SVC (Scalable Video Coding) so the server can drop video quality layers for congested clients instead of freezing the whole stream.
- Monitor QoE metrics (loss, jitter, RTT) continuously and dynamically switch media servers or routes (using a geographically distributed SFU/media-relay architecture) to reduce peak-hour congestion.

## Question 2: High DNS Lookup Time in a Multi-Continent Web Service

### 1. Reasons Behind the High DNS Resolution Time ( $\approx 800$ ms)

- Centralized/single-location DNS servers: if authoritative or resolver infrastructure is concentrated in one region, users far from that location incur significant propagation delay for every query.
- Deep recursive resolution chain: each lookup may need to traverse root  $\rightarrow$  TLD  $\rightarrow$  authoritative name servers if results are not cached, adding multiple round trips.
- Low cache hit ratio: aggressive or very low TTL values force frequent re-resolution instead of serving from local resolver cache.
- Lack of geographically distributed name servers: without Anycast, all queries funnel to the same physical server regardless of the user's location, adding queuing delay under load.
- No local DNS caching/pre-fetching at the client or ISP resolver level, so even repeat lookups pay the full round-trip cost.
- Overloaded or under-provisioned DNS servers during peak traffic, increasing queuing and processing delay.

### 2. Proposed DNS Architecture to Reduce Lookup Time Below 50 ms

**Anycast DNS:** Deploy the same authoritative DNS IP address from multiple geographically distributed points of presence (PoPs). BGP routing automatically directs each user's query to the nearest/lowest-latency PoP, eliminating long-haul propagation delay and providing built-in load distribution and failover.

**DNS Pre-fetching:** Have client applications and browsers resolve domain names for likely-needed resources in advance (e.g., while a user is on a login page, pre-resolve the domains of resources the homepage will need), so the resolution is already complete by the time the request is made.

**TTL Optimization:** Set TTL values that balance cache freshness against lookup frequency — moderate TTLs (e.g., 300–3600 seconds) for records that rarely change, allowing recursive resolvers and clients to serve repeat queries from cache without additional round trips, while keeping TTLs low enough to support failover when needed.

Combined effect: Anycast reduces the physical distance/latency per query, pre-fetching removes resolution from the critical path of page load, and TTL tuning maximizes cache reuse — together bringing typical lookups from ~800 ms down to well under 50 ms for cached/nearby queries.

### 3. Advantages and Limitations of the Proposed Architecture

#### Advantages:

- Performance: Anycast routes queries to the nearest server, sharply cutting round-trip time; pre-fetching hides latency from the user entirely.
- Scalability: multiple Anycast PoPs can each absorb a share of global query load, and the architecture scales horizontally by adding more PoPs.
- Availability: if one PoP fails, BGP automatically reroutes traffic to the next-nearest healthy PoP, providing seamless failover without manual DNS changes.

#### Limitations:

- Anycast routing depends on BGP convergence; misconfigured or unstable routing can occasionally send queries to a suboptimal or distant PoP.
- Overly long TTLs improve performance but slow down propagation of legitimate DNS changes (e.g., during failover or migration), while overly short TTLs increase load and reduce the performance benefit.
- Pre-fetching consumes extra client and network resources for resolutions that may not always be used, and requires careful implementation to avoid privacy/tracking concerns.
- Operating multiple Anycast PoPs increases infrastructure cost and operational complexity (monitoring, synchronization of DNS records across sites).

## Question 3: Centralized DNS Bottleneck for a Global Cloud Provider

### 4. Impact of Centralized DNS Architecture on Application Performance

Routing every global DNS query to a single primary server creates a single point of both latency and failure:

- High latency for distant users: users far from the primary server pay a large propagation delay on every uncached lookup (observed here as ~800 ms).
- Poor scalability under load: during promotional events, the sudden surge in simultaneous users creates queuing delay and possible timeouts as the single server becomes saturated.
- Single point of failure: if the primary server becomes unreachable or overloaded, all name resolution — and therefore all application access — fails globally, not just in one region.
- Cascading application-level delay: since DNS resolution is typically the first step before any HTTP/TCP connection, this bottleneck delays every downstream request, amplifying perceived slowness across the entire platform.

### 5. Design of a Distributed DNS Solution

A distributed, highly available DNS solution should combine the following components:

- Multiple Anycast-enabled authoritative DNS clusters deployed across continents (e.g., North America, Europe, Asia-Pacific), all announcing the same IP via BGP so each user reaches the nearest cluster.
- A tiered resolver hierarchy with geographically local recursive resolvers/edge caches close to end users, reducing dependence on any single origin server.

- Health-check-driven failover so that if one regional cluster becomes unhealthy, BGP/health checks redirect traffic to the next-nearest healthy cluster automatically.
- Load-balanced secondary/replica name servers within each region to absorb traffic spikes such as promotional events, using zone transfers (AXFR/IXFR) to keep records synchronized.
- Monitoring and analytics on query volume and latency per region to proactively scale capacity ahead of anticipated high-traffic events.

This design removes the single point of failure, distributes load across many servers instead of one, and places DNS resolution physically closer to users worldwide.

## 6. Influence of TTL, Anycast Routing, and DNS Caching on Performance and Fault Tolerance

**TTL values:** Moderate TTLs allow resolvers to cache answers and avoid repeated queries, reducing average latency and server load; however, TTLs that are too long delay the propagation of failover changes, while TTLs that are too short increase query volume and reduce the benefit of caching — the value must be tuned per record type (e.g., shorter TTLs for records tied to failover, longer for stable records).

**Anycast routing:** By allowing the same IP address to be served from many locations, Anycast reduces per-query latency (queries go to the nearest instance) and improves fault tolerance (BGP reroutes automatically around a failed or unreachable site) and naturally load-balances traffic across sites during demand spikes.

**DNS caching:** Caching at recursive resolvers, ISPs, and client devices means most repeat queries are answered locally without contacting the authoritative servers at all, dramatically cutting both latency and load on the origin infrastructure — but stale cached entries (due to overly long TTLs) can cause clients to keep contacting a failed server until the cache expires.

Together, these three mechanisms trade off responsiveness, freshness, and resilience; a well-tuned combination gives low latency in the common case while still allowing timely failover when something goes wrong.

## Question 4: TCP Latency Spike in an Electronic Stock Trading Platform

### 1. Three TCP-Related Factors Responsible for the Latency Increase

**a) Flow Control:** TCP's receive-window mechanism limits how much unacknowledged data a sender can transmit. Under a sudden surge of orders at market open, if the receiver (server) cannot process and acknowledge data fast enough, its advertised window shrinks, forcing the sender to pause transmission — introducing delay even though the network itself has capacity.

**b) Nagle's Algorithm:** Nagle's algorithm delays sending small packets so they can be coalesced into larger ones, reducing overhead for bulk transfers. For a trading system that sends many small, latency-sensitive order messages, this delay (which can combine badly with delayed ACKs, producing pauses of up to ~200 ms) is directly counterproductive, since each order needs to be sent immediately rather than batched.

**c) SYN Queue Limitations:** Each new TCP connection begins with a three-way handshake, and incoming SYN requests are held in a fixed-size SYN queue (backlog) at the server before being accepted. During the burst of connection attempts at market opening, if the SYN queue is too small or the accept rate too slow, new connection requests queue up or are dropped and retried, adding to the connection-establishment latency seen in the reported figures.

### 2. How Each Factor Affects High-Frequency Transaction Processing

- Flow control: causes intermittent stalls in already-open connections precisely when order volume is highest, meaning some orders sit buffered rather than being transmitted, directly increasing acknowledgment latency from the reported 1 ms baseline.

- Nagle's algorithm: delays the transmission of individual small order/acknowledgment messages while waiting to batch them, which is fundamentally at odds with a system where each message must be sent with minimal delay — this alone can account for tens of milliseconds of added latency per message.
- SYN queue limitations: increase the time needed to establish new sessions during the burst of market-open activity, so trading clients experience connection delays before they can even begin submitting orders, and repeated SYN retransmissions after drops further inflate the observed latency.

### 3. Recommended TCP Configuration Changes

- Disable Nagle's algorithm (set `TCP_NODELAY`) on all trading connections so small order messages are transmitted immediately rather than buffered for coalescing.
- Disable or tune delayed ACKs (avoid the classic Nagle + delayed-ACK interaction) so acknowledgments are also sent promptly rather than being held back.
- Increase the TCP receive/send window sizes (and enable window scaling) so flow control does not unnecessarily throttle throughput during high-order-volume bursts.
- Increase the SYN backlog queue size (e.g., tuning `somaxconn`/`tcp_max_syn_backlog``) and enable SYN cookies, so the server can absorb a burst of new connection requests at market open without dropping or delaying handshakes.
- Use persistent/pre-established (keep-alive) connections for known trading clients instead of opening a new TCP connection per session, avoiding repeated handshake overhead entirely.
- Consider a low-latency transport alternative for the most latency-sensitive paths (e.g., UDP-based proprietary protocols with application-level reliability, or kernel-bypass techniques) where regulatory and reliability requirements permit.
- Continuously monitor retransmission rates, RTT, and queue depths, and auto-scale server capacity ahead of known high-volume periods such as market open.

## Question 1: Video Conferencing Platform — Packet Loss, Jitter, and Delay

### 1. Analysis: Transport Protocol vs Application Layer

The problem described — 8% packet loss, 180 ms jitter, frozen video frames, and voice distortion during peak hours — is primarily caused by the choice and behaviour of the transport protocol, though the application layer's buffering strategy amplifies the visible effect.

If the platform uses TCP for real-time audio/video, TCP's reliability mechanisms (retransmission, in-order delivery, congestion control) are the root cause. When a packet is lost, TCP halts delivery of all subsequent data until the lost segment is retransmitted and acknowledged. Under 8% loss and high jitter, this creates head-of-line blocking, so frames arrive too late to be useful, causing the frozen-frame symptom described.

If UDP is already being used (as most real conferencing platforms do), the packet loss and jitter are largely a symptom of network-layer congestion at peak hours (shared bandwidth, queuing delay, router buffer overflow) rather than the protocol itself. However, the application layer is still responsible for how it copes with this loss — if the jitter buffer or packet-loss concealment logic is poorly tuned, the same underlying network conditions will manifest as worse distortion and freezing.

Conclusion: the underlying trigger is transport/network-layer congestion and loss, but the severity experienced by users is strongly shaped by the application layer's buffering and concealment strategy. Both layers must be examined together.

### 2. Justification Based on Real-Time Multimedia Requirements

Real-time multimedia communication has fundamentally different requirements from typical data transfer:

- Timeliness over completeness: a late packet is worse than a lost one, since stale audio/video frames are useless once the presentation deadline passes.
- Tolerance for loss: human perception can tolerate small amounts of packet loss (a brief glitch) far better than added delay.
- Low and predictable latency: interactive conversation requires end-to-end delay under roughly 150 ms to feel natural; jitter must be smoothed without adding excessive buffering delay.
- Continuous stream delivery: video/audio codecs expect a steady stream of samples; retransmission-based recovery disrupts this rhythm.

TCP's guarantees (in-order, lossless delivery) directly conflict with these requirements because retransmission and congestion-control back-off increase delay precisely when the network is already congested. UDP better matches these requirements because it delivers packets immediately without waiting for retransmission, leaving loss-handling and pacing to the application (via RTP/RTCP), which is the standard design used by real conferencing systems.

### 3. Recommended Protocol-Level and Application-Level Improvements

#### Protocol-level:

- Use UDP as the transport, carried over RTP/RTCP (or SRTP for security) instead of TCP, so late data is simply dropped rather than blocking the stream.
- Enable adaptive bitrate/congestion control designed for real-time media, such as Google Congestion Control (GCC) or WebRTC's built-in bandwidth estimation, so encoding rate adjusts to available bandwidth instead of overflowing the network.
- Apply Differentiated Services (DiffServ/QoS) marking (e.g., EF — Expedited Forwarding) so routers prioritize voice/video packets over best-effort traffic during peak congestion.
- Use Forward Error Correction (FEC) to reconstruct a percentage of lost packets without needing retransmission, which is well suited to an 8% loss rate.

#### Application-level:

- Implement an adaptive jitter buffer that dynamically resizes based on measured jitter (180 ms) rather than using a fixed buffer, balancing smoothness against added delay.
- Use packet-loss concealment (PLC) algorithms in the audio codec (e.g., Opus's built-in PLC) to mask short losses without perceptible artifacts.
- Enable simulcast/SVC (Scalable Video Coding) so the server can drop video quality layers for congested clients instead of freezing the whole stream.
- Monitor QoE metrics (loss, jitter, RTT) continuously and dynamically switch media servers or routes (using a geographically distributed SFU/media-relay architecture) to reduce peak-hour congestion.

## Question 2: High DNS Lookup Time in a Multi-Continent Web Service

### 1. Reasons Behind the High DNS Resolution Time ( $\approx 800$ ms)

- Centralized/single-location DNS servers: if authoritative or resolver infrastructure is concentrated in one region, users far from that location incur significant propagation delay for every query.
- Deep recursive resolution chain: each lookup may need to traverse root  $\rightarrow$  TLD  $\rightarrow$  authoritative name servers if results are not cached, adding multiple round trips.
- Low cache hit ratio: aggressive or very low TTL values force frequent re-resolution instead of serving from local resolver cache.

- Lack of geographically distributed name servers: without Anycast, all queries funnel to the same physical server regardless of the user's location, adding queuing delay under load.
- No local DNS caching/pre-fetching at the client or ISP resolver level, so even repeat lookups pay the full round-trip cost.
- Overloaded or under-provisioned DNS servers during peak traffic, increasing queuing and processing delay.

## 2. Proposed DNS Architecture to Reduce Lookup Time Below 50 ms

**Anycast DNS:** Deploy the same authoritative DNS IP address from multiple geographically distributed points of presence (PoPs). BGP routing automatically directs each user's query to the nearest/lowest-latency PoP, eliminating long-haul propagation delay and providing built-in load distribution and failover.

**DNS Pre-fetching:** Have client applications and browsers resolve domain names for likely-needed resources in advance (e.g., while a user is on a login page, pre-resolve the domains of resources the homepage will need), so the resolution is already complete by the time the request is made.

**TTL Optimization:** Set TTL values that balance cache freshness against lookup frequency — moderate TTLs (e.g., 300–3600 seconds) for records that rarely change, allowing recursive resolvers and clients to serve repeat queries from cache without additional round trips, while keeping TTLs low enough to support failover when needed.

Combined effect: Anycast reduces the physical distance/latency per query, pre-fetching removes resolution from the critical path of page load, and TTL tuning maximizes cache reuse — together bringing typical lookups from ~800 ms down to well under 50 ms for cached/nearby queries.

## 3. Advantages and Limitations of the Proposed Architecture

### Advantages:

- Performance: Anycast routes queries to the nearest server, sharply cutting round-trip time; pre-fetching hides latency from the user entirely.
- Scalability: multiple Anycast PoPs can each absorb a share of global query load, and the architecture scales horizontally by adding more PoPs.
- Availability: if one PoP fails, BGP automatically reroutes traffic to the next-nearest healthy PoP, providing seamless failover without manual DNS changes.

### Limitations:

- Anycast routing depends on BGP convergence; misconfigured or unstable routing can occasionally send queries to a suboptimal or distant PoP.
- Overly long TTLs improve performance but slow down propagation of legitimate DNS changes (e.g., during failover or migration), while overly short TTLs increase load and reduce the performance benefit.
- Pre-fetching consumes extra client and network resources for resolutions that may not always be used, and requires careful implementation to avoid privacy/tracking concerns.
- Operating multiple Anycast PoPs increases infrastructure cost and operational complexity (monitoring, synchronization of DNS records across sites).

## Question 3: Centralized DNS Bottleneck for a Global Cloud Provider

### 4. Impact of Centralized DNS Architecture on Application Performance

Routing every global DNS query to a single primary server creates a single point of both latency and failure:



- High latency for distant users: users far from the primary server pay a large propagation delay on every uncached lookup (observed here as ~800 ms).
- Poor scalability under load: during promotional events, the sudden surge in simultaneous users creates queuing delay and possible timeouts as the single server becomes saturated.
- Single point of failure: if the primary server becomes unreachable or overloaded, all name resolution — and therefore all application access — fails globally, not just in one region.
- Cascading application-level delay: since DNS resolution is typically the first step before any HTTP/TCP connection, this bottleneck delays every downstream request, amplifying perceived slowness across the entire platform.

## 5. Design of a Distributed DNS Solution

A distributed, highly available DNS solution should combine the following components:

- Multiple Anycast-enabled authoritative DNS clusters deployed across continents (e.g., North America, Europe, Asia-Pacific), all announcing the same IP via BGP so each user reaches the nearest cluster.
- A tiered resolver hierarchy with geographically local recursive resolvers/edge caches close to end users, reducing dependence on any single origin server.
- Health-check-driven failover so that if one regional cluster becomes unhealthy, BGP/health checks redirect traffic to the next-nearest healthy cluster automatically.
- Load-balanced secondary/replica name servers within each region to absorb traffic spikes such as promotional events, using zone transfers (AXFR/IXFR) to keep records synchronized.
- Monitoring and analytics on query volume and latency per region to proactively scale capacity ahead of anticipated high-traffic events.

This design removes the single point of failure, distributes load across many servers instead of one, and places DNS resolution physically closer to users worldwide.

## 6. Influence of TTL, Anycast Routing, and DNS Caching on Performance and Fault Tolerance

**TTL values:** Moderate TTLs allow resolvers to cache answers and avoid repeated queries, reducing average latency and server load; however, TTLs that are too long delay the propagation of failover changes, while TTLs that are too short increase query volume and reduce the benefit of caching — the value must be tuned per record type (e.g., shorter TTLs for records tied to failover, longer for stable records).

**Anycast routing:** By allowing the same IP address to be served from many locations, Anycast reduces per-query latency (queries go to the nearest instance) and improves fault tolerance (BGP reroutes automatically around a failed or unreachable site) and naturally load-balances traffic across sites during demand spikes.

**DNS caching:** Caching at recursive resolvers, ISPs, and client devices means most repeat queries are answered locally without contacting the authoritative servers at all, dramatically cutting both latency and load on the origin infrastructure — but stale cached entries (due to overly long TTLs) can cause clients to keep contacting a failed server until the cache expires.

Together, these three mechanisms trade off responsiveness, freshness, and resilience; a well-tuned combination gives low latency in the common case while still allowing timely failover when something goes wrong.

## Question 4: TCP Latency Spike in an Electronic Stock Trading Platform

### 1. Three TCP-Related Factors Responsible for the Latency Increase

**a) Flow Control:** TCP's receive-window mechanism limits how much unacknowledged data a sender can transmit. Under a sudden surge of orders at market open, if the receiver (server) cannot process and

acknowledge data fast enough, its advertised window shrinks, forcing the sender to pause transmission — introducing delay even though the network itself has capacity.

**b) Nagle's Algorithm:** Nagle's algorithm delays sending small packets so they can be coalesced into larger ones, reducing overhead for bulk transfers. For a trading system that sends many small, latency-sensitive order messages, this delay (which can combine badly with delayed ACKs, producing pauses of up to ~200 ms) is directly counterproductive, since each order needs to be sent immediately rather than batched.

**c) SYN Queue Limitations:** Each new TCP connection begins with a three-way handshake, and incoming SYN requests are held in a fixed-size SYN queue (backlog) at the server before being accepted. During the burst of connection attempts at market opening, if the SYN queue is too small or the accept rate too slow, new connection requests queue up or are dropped and retried, adding to the connection-establishment latency seen in the reported figures.

## 2. How Each Factor Affects High-Frequency Transaction Processing

- Flow control: causes intermittent stalls in already-open connections precisely when order volume is highest, meaning some orders sit buffered rather than being transmitted, directly increasing acknowledgment latency from the reported 1 ms baseline.
- Nagle's algorithm: delays the transmission of individual small order/acknowledgment messages while waiting to batch them, which is fundamentally at odds with a system where each message must be sent with minimal delay — this alone can account for tens of milliseconds of added latency per message.
- SYN queue limitations: increase the time needed to establish new sessions during the burst of market-open activity, so trading clients experience connection delays before they can even begin submitting orders, and repeated SYN retransmissions after drops further inflate the observed latency.

## 3. Recommended TCP Configuration Changes

- Disable Nagle's algorithm (set `TCP_NODELAY`) on all trading connections so small order messages are transmitted immediately rather than buffered for coalescing.
- Disable or tune delayed ACKs (avoid the classic Nagle + delayed-ACK interaction) so acknowledgments are also sent promptly rather than being held back.
- Increase the TCP receive/send window sizes (and enable window scaling) so flow control does not unnecessarily throttle throughput during high-order-volume bursts.
- Increase the SYN backlog queue size (e.g., tuning `somaxconn`/`tcp_max_syn_backlog``) and enable SYN cookies, so the server can absorb a burst of new connection requests at market open without dropping or delaying handshakes.
- Use persistent/pre-established (keep-alive) connections for known trading clients instead of opening a new TCP connection per session, avoiding repeated handshake overhead entirely.
- Consider a low-latency transport alternative for the most latency-sensitive paths (e.g., UDP-based proprietary protocols with application-level reliability, or kernel-bypass techniques) where regulatory and reliability requirements permit.
- Continuously monitor retransmission rates, RTT, and queue depths, and auto-scale server capacity ahead of known high-volume periods such as market open.

## Question 1: Video Conferencing Platform — Packet Loss, Jitter, and Delay

### 1. Analysis: Transport Protocol vs Application Layer

The problem described — 8% packet loss, 180 ms jitter, frozen video frames, and voice distortion during peak hours — is primarily caused by the choice and behaviour of the transport protocol, though the application layer's buffering strategy amplifies the visible effect.

If the platform uses TCP for real-time audio/video, TCP's reliability mechanisms (retransmission, in-order delivery, congestion control) are the root cause. When a packet is lost, TCP halts delivery of all subsequent data until the lost segment is retransmitted and acknowledged. Under 8% loss and high jitter, this creates head-of-line blocking, so frames arrive too late to be useful, causing the frozen-frame symptom described.

If UDP is already being used (as most real conferencing platforms do), the packet loss and jitter are largely a symptom of network-layer congestion at peak hours (shared bandwidth, queuing delay, router buffer overflow) rather than the protocol itself. However, the application layer is still responsible for how it copes with this loss — if the jitter buffer or packet-loss concealment logic is poorly tuned, the same underlying network conditions will manifest as worse distortion and freezing.

Conclusion: the underlying trigger is transport/network-layer congestion and loss, but the severity experienced by users is strongly shaped by the application layer's buffering and concealment strategy. Both layers must be examined together.

## 2. Justification Based on Real-Time Multimedia Requirements

Real-time multimedia communication has fundamentally different requirements from typical data transfer:

- Timeliness over completeness: a late packet is worse than a lost one, since stale audio/video frames are useless once the presentation deadline passes.
- Tolerance for loss: human perception can tolerate small amounts of packet loss (a brief glitch) far better than added delay.
- Low and predictable latency: interactive conversation requires end-to-end delay under roughly 150 ms to feel natural; jitter must be smoothed without adding excessive buffering delay.
- Continuous stream delivery: video/audio codecs expect a steady stream of samples; retransmission-based recovery disrupts this rhythm.

TCP's guarantees (in-order, lossless delivery) directly conflict with these requirements because retransmission and congestion-control back-off increase delay precisely when the network is already congested. UDP better matches these requirements because it delivers packets immediately without waiting for retransmission, leaving loss-handling and pacing to the application (via RTP/RTCP), which is the standard design used by real conferencing systems.

## 3. Recommended Protocol-Level and Application-Level Improvements

### Protocol-level:

- Use UDP as the transport, carried over RTP/RTCP (or SRTP for security) instead of TCP, so late data is simply dropped rather than blocking the stream.
- Enable adaptive bitrate/congestion control designed for real-time media, such as Google Congestion Control (GCC) or WebRTC's built-in bandwidth estimation, so encoding rate adjusts to available bandwidth instead of overflowing the network.
- Apply Differentiated Services (DiffServ/QoS) marking (e.g., EF — Expedited Forwarding) so routers prioritize voice/video packets over best-effort traffic during peak congestion.
- Use Forward Error Correction (FEC) to reconstruct a percentage of lost packets without needing retransmission, which is well suited to an 8% loss rate.

### Application-level:

- Implement an adaptive jitter buffer that dynamically resizes based on measured jitter (180 ms) rather than using a fixed buffer, balancing smoothness against added delay.
- Use packet-loss concealment (PLC) algorithms in the audio codec (e.g., Opus's built-in PLC) to mask short losses without perceptible artifacts.
- Enable simulcast/SVC (Scalable Video Coding) so the server can drop video quality layers for congested clients instead of freezing the whole stream.

- Monitor QoE metrics (loss, jitter, RTT) continuously and dynamically switch media servers or routes (using a geographically distributed SFU/media-relay architecture) to reduce peak-hour congestion.

## Question 2: High DNS Lookup Time in a Multi-Continent Web Service

### 1. Reasons Behind the High DNS Resolution Time ( $\approx 800$ ms)

- Centralized/single-location DNS servers: if authoritative or resolver infrastructure is concentrated in one region, users far from that location incur significant propagation delay for every query.
- Deep recursive resolution chain: each lookup may need to traverse root  $\rightarrow$  TLD  $\rightarrow$  authoritative name servers if results are not cached, adding multiple round trips.
- Low cache hit ratio: aggressive or very low TTL values force frequent re-resolution instead of serving from local resolver cache.
- Lack of geographically distributed name servers: without Anycast, all queries funnel to the same physical server regardless of the user's location, adding queuing delay under load.
- No local DNS caching/pre-fetching at the client or ISP resolver level, so even repeat lookups pay the full round-trip cost.
- Overloaded or under-provisioned DNS servers during peak traffic, increasing queuing and processing delay.

### 2. Proposed DNS Architecture to Reduce Lookup Time Below 50 ms

**Anycast DNS:** Deploy the same authoritative DNS IP address from multiple geographically distributed points of presence (PoPs). BGP routing automatically directs each user's query to the nearest/lowest-latency PoP, eliminating long-haul propagation delay and providing built-in load distribution and failover.

**DNS Pre-fetching:** Have client applications and browsers resolve domain names for likely-needed resources in advance (e.g., while a user is on a login page, pre-resolve the domains of resources the homepage will need), so the resolution is already complete by the time the request is made.

**TTL Optimization:** Set TTL values that balance cache freshness against lookup frequency — moderate TTLs (e.g., 300–3600 seconds) for records that rarely change, allowing recursive resolvers and clients to serve repeat queries from cache without additional round trips, while keeping TTLs low enough to support failover when needed.

Combined effect: Anycast reduces the physical distance/latency per query, pre-fetching removes resolution from the critical path of page load, and TTL tuning maximizes cache reuse — together bringing typical lookups from  $\sim 800$  ms down to well under 50 ms for cached/nearby queries.

### 3. Advantages and Limitations of the Proposed Architecture

#### Advantages:

- Performance: Anycast routes queries to the nearest server, sharply cutting round-trip time; pre-fetching hides latency from the user entirely.
- Scalability: multiple Anycast PoPs can each absorb a share of global query load, and the architecture scales horizontally by adding more PoPs.
- Availability: if one PoP fails, BGP automatically reroutes traffic to the next-nearest healthy PoP, providing seamless failover without manual DNS changes.

#### Limitations:

- Anycast routing depends on BGP convergence; misconfigured or unstable routing can occasionally send queries to a suboptimal or distant PoP.

- Overly long TTLs improve performance but slow down propagation of legitimate DNS changes (e.g., during failover or migration), while overly short TTLs increase load and reduce the performance benefit.
- Pre-fetching consumes extra client and network resources for resolutions that may not always be used, and requires careful implementation to avoid privacy/tracking concerns.
- Operating multiple Anycast PoPs increases infrastructure cost and operational complexity (monitoring, synchronization of DNS records across sites).

## Question 3: Centralized DNS Bottleneck for a Global Cloud Provider

### 4. Impact of Centralized DNS Architecture on Application Performance

Routing every global DNS query to a single primary server creates a single point of both latency and failure:

- High latency for distant users: users far from the primary server pay a large propagation delay on every uncached lookup (observed here as ~800 ms).
- Poor scalability under load: during promotional events, the sudden surge in simultaneous users creates queuing delay and possible timeouts as the single server becomes saturated.
- Single point of failure: if the primary server becomes unreachable or overloaded, all name resolution — and therefore all application access — fails globally, not just in one region.
- Cascading application-level delay: since DNS resolution is typically the first step before any HTTP/TCP connection, this bottleneck delays every downstream request, amplifying perceived slowness across the entire platform.

### 5. Design of a Distributed DNS Solution

A distributed, highly available DNS solution should combine the following components:

- Multiple Anycast-enabled authoritative DNS clusters deployed across continents (e.g., North America, Europe, Asia-Pacific), all announcing the same IP via BGP so each user reaches the nearest cluster.
- A tiered resolver hierarchy with geographically local recursive resolvers/edge caches close to end users, reducing dependence on any single origin server.
- Health-check-driven failover so that if one regional cluster becomes unhealthy, BGP/health checks redirect traffic to the next-nearest healthy cluster automatically.
- Load-balanced secondary/replica name servers within each region to absorb traffic spikes such as promotional events, using zone transfers (AXFR/IXFR) to keep records synchronized.
- Monitoring and analytics on query volume and latency per region to proactively scale capacity ahead of anticipated high-traffic events.

This design removes the single point of failure, distributes load across many servers instead of one, and places DNS resolution physically closer to users worldwide.

### 6. Influence of TTL, Anycast Routing, and DNS Caching on Performance and Fault Tolerance

**TTL values:** Moderate TTLs allow resolvers to cache answers and avoid repeated queries, reducing average latency and server load; however, TTLs that are too long delay the propagation of failover changes, while TTLs that are too short increase query volume and reduce the benefit of caching — the value must be tuned per record type (e.g., shorter TTLs for records tied to failover, longer for stable records).

**Anycast routing:** By allowing the same IP address to be served from many locations, Anycast reduces per-query latency (queries go to the nearest instance) and improves fault tolerance (BGP reroutes automatically around a failed or unreachable site) and naturally load-balances traffic across sites during demand spikes.

**DNS caching:** Caching at recursive resolvers, ISPs, and client devices means most repeat queries are answered locally without contacting the authoritative servers at all, dramatically cutting both latency and load on the origin infrastructure — but stale cached entries (due to overly long TTLs) can cause clients to keep contacting a failed server until the cache expires.

Together, these three mechanisms trade off responsiveness, freshness, and resilience; a well-tuned combination gives low latency in the common case while still allowing timely failover when something goes wrong.

## Question 4: TCP Latency Spike in an Electronic Stock Trading Platform

### 1. Three TCP-Related Factors Responsible for the Latency Increase

**a) Flow Control:** TCP's receive-window mechanism limits how much unacknowledged data a sender can transmit. Under a sudden surge of orders at market open, if the receiver (server) cannot process and acknowledge data fast enough, its advertised window shrinks, forcing the sender to pause transmission — introducing delay even though the network itself has capacity.

**b) Nagle's Algorithm:** Nagle's algorithm delays sending small packets so they can be coalesced into larger ones, reducing overhead for bulk transfers. For a trading system that sends many small, latency-sensitive order messages, this delay (which can combine badly with delayed ACKs, producing pauses of up to ~200 ms) is directly counterproductive, since each order needs to be sent immediately rather than batched.

**c) SYN Queue Limitations:** Each new TCP connection begins with a three-way handshake, and incoming SYN requests are held in a fixed-size SYN queue (backlog) at the server before being accepted. During the burst of connection attempts at market opening, if the SYN queue is too small or the accept rate too slow, new connection requests queue up or are dropped and retried, adding to the connection-establishment latency seen in the reported figures.

### 2. How Each Factor Affects High-Frequency Transaction Processing

- Flow control: causes intermittent stalls in already-open connections precisely when order volume is highest, meaning some orders sit buffered rather than being transmitted, directly increasing acknowledgment latency from the reported 1 ms baseline.
- Nagle's algorithm: delays the transmission of individual small order/acknowledgment messages while waiting to batch them, which is fundamentally at odds with a system where each message must be sent with minimal delay — this alone can account for tens of milliseconds of added latency per message.
- SYN queue limitations: increase the time needed to establish new sessions during the burst of market-open activity, so trading clients experience connection delays before they can even begin submitting orders, and repeated SYN retransmissions after drops further inflate the observed latency.

### 3. Recommended TCP Configuration Changes

- Disable Nagle's algorithm (set `TCP_NODELAY`) on all trading connections so small order messages are transmitted immediately rather than buffered for coalescing.
- Disable or tune delayed ACKs (avoid the classic Nagle + delayed-ACK interaction) so acknowledgments are also sent promptly rather than being held back.
- Increase the TCP receive/send window sizes (and enable window scaling) so flow control does not unnecessarily throttle throughput during high-order-volume bursts.
- Increase the SYN backlog queue size (e.g., tuning `somaxconn`/`tcp_max_syn_backlog``) and enable SYN cookies, so the server can absorb a burst of new connection requests at market open without dropping or delaying handshakes.
- Use persistent/pre-established (keep-alive) connections for known trading clients instead of opening a new TCP connection per session, avoiding repeated handshake overhead entirely.



- Consider a low-latency transport alternative for the most latency-sensitive paths (e.g., UDP-based proprietary protocols with application-level reliability, or kernel-bypass techniques) where regulatory and reliability requirements permit.
- Continuously monitor retransmission rates, RTT, and queue depths, and auto-scale server capacity ahead of known high-volume periods such as market open.

## Question 1: Video Conferencing Platform — Packet Loss, Jitter, and Delay

### 1. Analysis: Transport Protocol vs Application Layer

The problem described — 8% packet loss, 180 ms jitter, frozen video frames, and voice distortion during peak hours — is primarily caused by the choice and behaviour of the transport protocol, though the application layer's buffering strategy amplifies the visible effect.

If the platform uses TCP for real-time audio/video, TCP's reliability mechanisms (retransmission, in-order delivery, congestion control) are the root cause. When a packet is lost, TCP halts delivery of all subsequent data until the lost segment is retransmitted and acknowledged. Under 8% loss and high jitter, this creates head-of-line blocking, so frames arrive too late to be useful, causing the frozen-frame symptom described.

If UDP is already being used (as most real conferencing platforms do), the packet loss and jitter are largely a symptom of network-layer congestion at peak hours (shared bandwidth, queuing delay, router buffer overflow) rather than the protocol itself. However, the application layer is still responsible for how it copes with this loss — if the jitter buffer or packet-loss concealment logic is poorly tuned, the same underlying network conditions will manifest as worse distortion and freezing.

Conclusion: the underlying trigger is transport/network-layer congestion and loss, but the severity experienced by users is strongly shaped by the application layer's buffering and concealment strategy. Both layers must be examined together.

### 2. Justification Based on Real-Time Multimedia Requirements

Real-time multimedia communication has fundamentally different requirements from typical data transfer:

- Timeliness over completeness: a late packet is worse than a lost one, since stale audio/video frames are useless once the presentation deadline passes.
- Tolerance for loss: human perception can tolerate small amounts of packet loss (a brief glitch) far better than added delay.
- Low and predictable latency: interactive conversation requires end-to-end delay under roughly 150 ms to feel natural; jitter must be smoothed without adding excessive buffering delay.
- Continuous stream delivery: video/audio codecs expect a steady stream of samples; retransmission-based recovery disrupts this rhythm.

TCP's guarantees (in-order, lossless delivery) directly conflict with these requirements because retransmission and congestion-control back-off increase delay precisely when the network is already congested. UDP better matches these requirements because it delivers packets immediately without waiting for retransmission, leaving loss-handling and pacing to the application (via RTP/RTCP), which is the standard design used by real conferencing systems.

### 3. Recommended Protocol-Level and Application-Level Improvements

#### Protocol-level:

- Use UDP as the transport, carried over RTP/RTCP (or SRTP for security) instead of TCP, so late data is simply dropped rather than blocking the stream.

- Enable adaptive bitrate/congestion control designed for real-time media, such as Google Congestion Control (GCC) or WebRTC's built-in bandwidth estimation, so encoding rate adjusts to available bandwidth instead of overflowing the network.
- Apply Differentiated Services (DiffServ/QoS) marking (e.g., EF — Expedited Forwarding) so routers prioritize voice/video packets over best-effort traffic during peak congestion.
- Use Forward Error Correction (FEC) to reconstruct a percentage of lost packets without needing retransmission, which is well suited to an 8% loss rate.

### Application-level:

- Implement an adaptive jitter buffer that dynamically resizes based on measured jitter (180 ms) rather than using a fixed buffer, balancing smoothness against added delay.
- Use packet-loss concealment (PLC) algorithms in the audio codec (e.g., Opus's built-in PLC) to mask short losses without perceptible artifacts.
- Enable simulcast/SVC (Scalable Video Coding) so the server can drop video quality layers for congested clients instead of freezing the whole stream.
- Monitor QoE metrics (loss, jitter, RTT) continuously and dynamically switch media servers or routes (using a geographically distributed SFU/media-relay architecture) to reduce peak-hour congestion.

## Question 2: High DNS Lookup Time in a Multi-Continent Web Service

### 1. Reasons Behind the High DNS Resolution Time ( $\approx 800$ ms)

- Centralized/single-location DNS servers: if authoritative or resolver infrastructure is concentrated in one region, users far from that location incur significant propagation delay for every query.
- Deep recursive resolution chain: each lookup may need to traverse root  $\rightarrow$  TLD  $\rightarrow$  authoritative name servers if results are not cached, adding multiple round trips.
- Low cache hit ratio: aggressive or very low TTL values force frequent re-resolution instead of serving from local resolver cache.
- Lack of geographically distributed name servers: without Anycast, all queries funnel to the same physical server regardless of the user's location, adding queuing delay under load.
- No local DNS caching/pre-fetching at the client or ISP resolver level, so even repeat lookups pay the full round-trip cost.
- Overloaded or under-provisioned DNS servers during peak traffic, increasing queuing and processing delay.

### 2. Proposed DNS Architecture to Reduce Lookup Time Below 50 ms

**Anycast DNS:** Deploy the same authoritative DNS IP address from multiple geographically distributed points of presence (PoPs). BGP routing automatically directs each user's query to the nearest/lowest-latency PoP, eliminating long-haul propagation delay and providing built-in load distribution and failover.

**DNS Pre-fetching:** Have client applications and browsers resolve domain names for likely-needed resources in advance (e.g., while a user is on a login page, pre-resolve the domains of resources the homepage will need), so the resolution is already complete by the time the request is made.

**TTL Optimization:** Set TTL values that balance cache freshness against lookup frequency — moderate TTLs (e.g., 300–3600 seconds) for records that rarely change, allowing recursive resolvers and clients to serve repeat queries from cache without additional round trips, while keeping TTLs low enough to support failover when needed.

Combined effect: Anycast reduces the physical distance/latency per query, pre-fetching removes resolution from the critical path of page load, and TTL tuning maximizes cache reuse — together bringing typical lookups from ~800 ms down to well under 50 ms for cached/nearby queries.

### 3. Advantages and Limitations of the Proposed Architecture

#### Advantages:

- Performance: Anycast routes queries to the nearest server, sharply cutting round-trip time; pre-fetching hides latency from the user entirely.
- Scalability: multiple Anycast PoPs can each absorb a share of global query load, and the architecture scales horizontally by adding more PoPs.
- Availability: if one PoP fails, BGP automatically reroutes traffic to the next-nearest healthy PoP, providing seamless failover without manual DNS changes.

#### Limitations:

- Anycast routing depends on BGP convergence; misconfigured or unstable routing can occasionally send queries to a suboptimal or distant PoP.
- Overly long TTLs improve performance but slow down propagation of legitimate DNS changes (e.g., during failover or migration), while overly short TTLs increase load and reduce the performance benefit.
- Pre-fetching consumes extra client and network resources for resolutions that may not always be used, and requires careful implementation to avoid privacy/tracking concerns.
- Operating multiple Anycast PoPs increases infrastructure cost and operational complexity (monitoring, synchronization of DNS records across sites).

## Question 3: Centralized DNS Bottleneck for a Global Cloud Provider

### 4. Impact of Centralized DNS Architecture on Application Performance

Routing every global DNS query to a single primary server creates a single point of both latency and failure:

- High latency for distant users: users far from the primary server pay a large propagation delay on every uncached lookup (observed here as ~800 ms).
- Poor scalability under load: during promotional events, the sudden surge in simultaneous users creates queuing delay and possible timeouts as the single server becomes saturated.
- Single point of failure: if the primary server becomes unreachable or overloaded, all name resolution — and therefore all application access — fails globally, not just in one region.
- Cascading application-level delay: since DNS resolution is typically the first step before any HTTP/TCP connection, this bottleneck delays every downstream request, amplifying perceived slowness across the entire platform.

### 5. Design of a Distributed DNS Solution

A distributed, highly available DNS solution should combine the following components:

- Multiple Anycast-enabled authoritative DNS clusters deployed across continents (e.g., North America, Europe, Asia-Pacific), all announcing the same IP via BGP so each user reaches the nearest cluster.
- A tiered resolver hierarchy with geographically local recursive resolvers/edge caches close to end users, reducing dependence on any single origin server.

- Health-check-driven failover so that if one regional cluster becomes unhealthy, BGP/health checks redirect traffic to the next-nearest healthy cluster automatically.
- Load-balanced secondary/replica name servers within each region to absorb traffic spikes such as promotional events, using zone transfers (AXFR/IXFR) to keep records synchronized.
- Monitoring and analytics on query volume and latency per region to proactively scale capacity ahead of anticipated high-traffic events.

This design removes the single point of failure, distributes load across many servers instead of one, and places DNS resolution physically closer to users worldwide.

## 6. Influence of TTL, Anycast Routing, and DNS Caching on Performance and Fault Tolerance

**TTL values:** Moderate TTLs allow resolvers to cache answers and avoid repeated queries, reducing average latency and server load; however, TTLs that are too long delay the propagation of failover changes, while TTLs that are too short increase query volume and reduce the benefit of caching — the value must be tuned per record type (e.g., shorter TTLs for records tied to failover, longer for stable records).

**Anycast routing:** By allowing the same IP address to be served from many locations, Anycast reduces per-query latency (queries go to the nearest instance) and improves fault tolerance (BGP reroutes automatically around a failed or unreachable site) and naturally load-balances traffic across sites during demand spikes.

**DNS caching:** Caching at recursive resolvers, ISPs, and client devices means most repeat queries are answered locally without contacting the authoritative servers at all, dramatically cutting both latency and load on the origin infrastructure — but stale cached entries (due to overly long TTLs) can cause clients to keep contacting a failed server until the cache expires.

Together, these three mechanisms trade off responsiveness, freshness, and resilience; a well-tuned combination gives low latency in the common case while still allowing timely failover when something goes wrong.

## Question 4: TCP Latency Spike in an Electronic Stock Trading Platform

### 1. Three TCP-Related Factors Responsible for the Latency Increase

**a) Flow Control:** TCP's receive-window mechanism limits how much unacknowledged data a sender can transmit. Under a sudden surge of orders at market open, if the receiver (server) cannot process and acknowledge data fast enough, its advertised window shrinks, forcing the sender to pause transmission — introducing delay even though the network itself has capacity.

**b) Nagle's Algorithm:** Nagle's algorithm delays sending small packets so they can be coalesced into larger ones, reducing overhead for bulk transfers. For a trading system that sends many small, latency-sensitive order messages, this delay (which can combine badly with delayed ACKs, producing pauses of up to ~200 ms) is directly counterproductive, since each order needs to be sent immediately rather than batched.

**c) SYN Queue Limitations:** Each new TCP connection begins with a three-way handshake, and incoming SYN requests are held in a fixed-size SYN queue (backlog) at the server before being accepted. During the burst of connection attempts at market opening, if the SYN queue is too small or the accept rate too slow, new connection requests queue up or are dropped and retried, adding to the connection-establishment latency seen in the reported figures.

### 2. How Each Factor Affects High-Frequency Transaction Processing

- Flow control: causes intermittent stalls in already-open connections precisely when order volume is highest, meaning some orders sit buffered rather than being transmitted, directly increasing acknowledgment latency from the reported 1 ms baseline.

- Nagle's algorithm: delays the transmission of individual small order/acknowledgment messages while waiting to batch them, which is fundamentally at odds with a system where each message must be sent with minimal delay — this alone can account for tens of milliseconds of added latency per message.
- SYN queue limitations: increase the time needed to establish new sessions during the burst of market-open activity, so trading clients experience connection delays before they can even begin submitting orders, and repeated SYN retransmissions after drops further inflate the observed latency.

### 3. Recommended TCP Configuration Changes

- Disable Nagle's algorithm (set `TCP_NODELAY`) on all trading connections so small order messages are transmitted immediately rather than buffered for coalescing.
- Disable or tune delayed ACKs (avoid the classic Nagle + delayed-ACK interaction) so acknowledgments are also sent promptly rather than being held back.
- Increase the TCP receive/send window sizes (and enable window scaling) so flow control does not unnecessarily throttle throughput during high-order-volume bursts.
- Increase the SYN backlog queue size (e.g., tuning `somaxconn`/`tcp_max_syn_backlog``) and enable SYN cookies, so the server can absorb a burst of new connection requests at market open without dropping or delaying handshakes.
- Use persistent/pre-established (keep-alive) connections for known trading clients instead of opening a new TCP connection per session, avoiding repeated handshake overhead entirely.
- Consider a low-latency transport alternative for the most latency-sensitive paths (e.g., UDP-based proprietary protocols with application-level reliability, or kernel-bypass techniques) where regulatory and reliability requirements permit.
- Continuously monitor retransmission rates, RTT, and queue depths, and auto-scale server capacity ahead of known high-volume periods such as market open.